

DSA COACH (TREE)

Project Scope and Implementation Document

Team name: Embed to conquer

Enterprise-Oriented Solution Scope, Architecture, Delivery Plan, and Agent Framework Evaluation

Document Date: 13 August 2026

Project Domain: Data Structures and Algorithms - Tree Data Structures

Frontend Baseline: Next.js 16.3.0 / React 19.2.8 / Auth.js 5 beta

Target Backend: Python / FastAPI / PostgreSQL / pgvector / BM25 / Local-first LLM

Members:

1. Sumit Panda (25C216A15)
2. Sunil Kumar Panda (25C216A10)
3. Samarjeet Sahoo (25C219A13)
4. Abhijeet Gouda (25C207A03)

Document Control

Field	Value
Project	DSA Coach (Tree)
Document	Scope and Implementation Plan
Version	1.0
Date	13 August 2026
Status	Implementation Planning / Enterprise Review
Primary Source	BACKEND_SPECIFICATION_CODEX.md
Frontend Source	treetutor-ai.zip

Executive Summary

DSA Coach (Tree) is the project, with TreeTutor AI as the existing frontend application. The platform is a specialized GenAI-enabled learning system focused initially on Data Structures and Algorithms (DSA), with Tree Data Structures as the first domain. The target educational workflow is to learn concepts through the RAG Agent, evaluate and improve code through the Step Evaluator, practice validated Tree problems, receive feedback, and improve over time. The supplied frontend is a Next.js application with dashboard, tutor, debugger, practice, and concept-library routes, while the current data/API layer is mock-based.

The implementation scope is to replace the mock data/service layer with a scalable, local-first Python backend while preserving the frontend contracts. The target backend is FastAPI with PostgreSQL as the primary database and pgvector for semantic retrieval. Retrieval must support true BM25, vector retrieval, and hybrid retrieval using Reciprocal Rank Fusion (RRF). A dedicated Supervisor Agent will route requests to specialized RAG Agent, Step Evaluator, and Practice Agent workflows. BM25, vector, and hybrid are selectable retrieval strategies; hybrid is the only mode that executes both BM25 and vector retrieval followed by RRF. Model selection remains configurable and local/free inference through Ollama is preferred.

The implementation plan intentionally follows a staged delivery approach: inspect the repository first, establish backend contracts and database foundations, implement ingestion and retrieval, introduce the agent layer, integrate APIs with the existing frontend, then add testing, observability, security hardening, and incremental future capabilities. The first meaningful milestone is the complete RAG Agent path through POST /api/chat; Step Evaluator, Practice, Conversations, Concepts, and Progress are subsequently delivered incrementally.

Contents

1. Project Definition
2. Current-State Assessment
3. Scope of the Project
4. Target Architecture
5. Implementation Strategy and Principles
6. Step-by-Step Implementation Plan

7. Agentic Framework Evaluation: LangGraph vs CrewAI

8. Recommended Framework for DSA Coach (Tree)

9. Testing and Quality Assurance

10. Security, Governance, and Observability

11. Deployment and Operational Readiness

12. Current vs Future Roadmap

13. Definition of Done and Acceptance Criteria

14. Risks, Constraints, and Mitigations

15. Conclusion

Appendix A - Example API and Agent Contracts

Appendix B - Reference Sources

1. Project Definition

Project Name: DSA Coach (Tree)

Project Type: GenAI-powered educational platform for Data Structures and Algorithms.

Initial Learning Domain: Tree Data Structures.

Primary Objective: provide grounded, role-specific AI assistance for learning, debugging, practicing, and feedback while retaining a clean architecture that can be extended to additional DSA domains.

1.1 Product Vision

LEARN -> RAG Agent -> UNDERSTAND CODE -> Step Evaluator -> PRACTICE -> Tree DSA Problems -> GET
FEEDBACK -> IMPROVE

The platform is not intended to behave as a generic chatbot. The backend is a specialized educational GenAI backend with distinct responsibilities for tutoring, debugging, practice, retrieval, memory, progress, and model orchestration.

1.2 Frontend Baseline

Area	Current implementation
Framework	Next.js 16.3.0
UI runtime	React 19.2.8
Authentication	NextAuth/Auth.js 5 beta scaffold
Language	JavaScript
UI styling	Tailwind CSS + shadcn-style components
Icons	Lucide React
API abstraction	lib/api.js
Current data source	lib/mock-data.js / mock service responses

2. Current-State Assessment

The supplied frontend establishes the user experience and the contracts that the backend must support. The backend implementation specification explicitly treats the existing frontend contract as the source of truth for initial compatibility.

2.1 Application Routes and User Journeys

Route	Primary capability	Backend dependency
/	Landing page	Static / frontend
/login	Authentication entry	NextAuth/Auth.js
/dashboard	Learning dashboard	Progress, activity, user data
/dashboard/tutor	Tree Tutor Chat	POST /api/chat + RAG Agent + conversation memory
/debugger	Code Debugger	POST /api/debug + Step Evaluator

Route	Primary capability	Backend dependency
/practice	Problem list and filtering	GET /api/problems
/practice/[id]	Problem solving and submission	GET /api/problems/{id} + POST /api/problems/{id}/submit
/concepts	Concept library	GET /api/concepts
/concepts/[slug]	Concept detail	GET /api/concepts/{slug}

2.2 Existing API Abstraction

```

sendMessage(message, history = [])
debugCode(code, language)
fetchProblems()
submitSolution(problemId, code, language)
fetchConcepts()
fetchConversation(id)

```

The frontend currently isolates these calls in lib/api.js. This is a favorable integration boundary because the mock implementation can be replaced by HTTP calls without forcing widespread changes to consuming components.

2.3 Existing UI Components Relevant to Backend Integration

Component area	Observed responsibility
Chat	ChatWindow, ChatMessage, AIMessage, UserMessage, ChatInput, CodeBlock, TypingIndicator
Debugger	CodeEditor, DebugPanel, ErrorCard
Practice	ProblemFilters, ProblemCard, ProblemDescription, TestResult
Concepts	ConceptCard, ConceptContent
Tree visualization	TreeVisualizer, TreeNode
Dashboard	StatsCards, ProgressOverview, RecentActivity
Layout	AppShell, Header, Sidebar, MobileSidebar

2.4 Current-State Gap

- AI chat is currently backed by mock responses and must be connected to the Python RAG/agent pipeline.
- Step evaluation is currently backed by a mock result and must be routed to the Step Evaluator Agent.
- Practice problems and results are currently sourced from mock data and must be moved to backend-backed problem data and, when safe execution exists, evaluation.
- Concepts are currently static/mock and should become backend-backed when content is dynamic.
- Dashboard metrics and recent activity are mock and should progressively come from PostgreSQL-backed user progress and activity.
- Authentication exists as a frontend scaffold, but secure verification between NextAuth/Auth.js and Python must be based on the actual production auth configuration before finalizing the mechanism.

3. Scope of the Project

3.1 Why This Project Is Being Implemented

The project is intended to move DSA Coach (Tree) from a frontend prototype using mock data to a production-oriented GenAI learning architecture. TreeTutor AI remains the existing frontend application. The primary need is not merely to add an LLM; it is to establish a reliable educational backend that can ground responses in a controlled Tree DSA knowledge base, route requests to specialized agents, preserve conversational context, and expose stable APIs to the existing frontend.

Retrieval policy: BM25, vector, and hybrid are alternative request-level modes. The Supervisor Agent or RAG Agent selects the mode; only the hybrid strategy invokes BM25 + vector retrieval and RRF fusion.

3.2 What the Current Scope Includes

- Python backend using FastAPI unless an existing functioning API framework in the repository should be retained.
- PostgreSQL as the primary application database.
- pgvector inside PostgreSQL for vector retrieval; no separate vector database for version 1.
- true BM25 retrieval using pg_search/ParadeDB when available and appropriate.
- Selectable retrieval modes: BM25, vector, or hybrid. Hybrid combines BM25 and vector results with RRF and is executed only when the hybrid mode is selected.
- Document ingestion with cleaning, heading-aware/semantic chunking, embeddings, and storage.
- Configurable local-first embedding and LLM providers, with Ollama as the default local inference path where feasible.
- A dedicated Supervisor Agent and three specialized agents: RAG Agent, Step Evaluator, and Practice Agent.
- Pydantic-based request/response schemas and normalized retrieval contracts.
- Conversation storage in PostgreSQL and optional summary memory when conversations become sufficiently long.
- Initial API surface required by the frontend.
- Authentication integration with the existing NextAuth/Auth.js approach rather than a second Python login system.
- Security controls, predictable error contracts, testing, logging/observability, and configuration documentation.

3.3 Current Functional Scope - Version 1

Capability	Version 1 scope	Success outcome
RAG Agent	RAG-grounded DSA teaching with selectable BM25, vector, or hybrid retrieval	POST /api/chat produces structured, grounded responses
Knowledge ingestion	Tree DSA documents -> chunks -> embeddings/indexes	Reproducible ingestion command and queryable knowledge base
Retrieval	BM25 + vector + hybrid/RRF, selected per request	Normalized RetrievalResult contract; hybrid uses RRF only when selected

Capability	Version 1 scope	Success outcome
Supervisor	Intent classification and delegation	Correct routing to specialized agent
Step Evaluator	Static code/DSA evaluation; no unsafe execution	Compatible Step Evaluator response through current /api/debug contract
Practice	Retrieve validated problems; generation where appropriate	Compatible problem contract and submission contract
Concepts	Concept list and detail API	Frontend can replace mock concept data
Conversation memory	Persistent messages; optional summaries	Conversation can be reopened
Progress	Useful initial metrics	Dashboard can progressively replace mocks

3.4 Future Scope

- Additional DSA tutors: Graph, Linked List, Stack/Queue, Sorting, Dynamic Programming and other domains.
- Safe code execution through an isolated execution service/container with CPU, memory, and time controls.
- Streaming chat responses after synchronous chat is stable.
- Personalized problem selection using user progress and weak-topic signals.
- Conversation summarization and richer long-term memory where justified by product usage.
- Optional retrieval reranking after measured retrieval evaluation demonstrates that RRF is insufficient.
- Tree visualization payloads generated as structured backend data for existing frontend rendering.
- Advanced evaluation, analytics, and governance for production model quality.
- Further integrations and deployment scaling only when measured workload requires them.

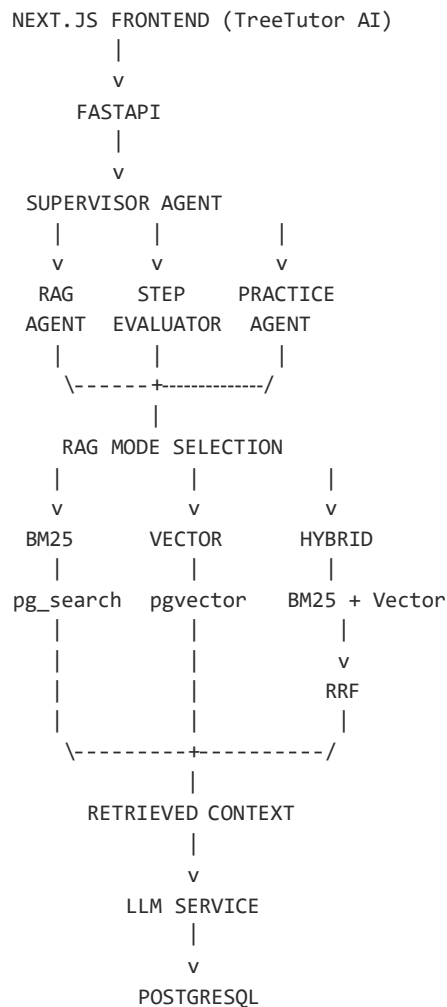
3.5 Explicitly Out of Scope for the Initial Release

- A second primary database or vector database such as MongoDB, FAISS, Pinecone, Chroma, Qdrant, Weaviate, or Milvus.
- Redis, Kafka, Celery, a graph database, distributed worker infrastructure, or model-serving clusters without a demonstrated requirement.
- A production code-execution sandbox unless an existing implementation is discovered or separately approved.
- A learned reranker in version 1.
- A separate Python authentication provider.
- Large automatic model downloads at application startup.
- Complex recommendation or machine-learning personalization systems.
- Implementation of every future agent/tool merely for extensibility.

3.6 Scope Principles

- Inspect before replacing: existing backend components, models, RAG utilities, agent code, and database approaches must be assessed before implementation.
- API compatibility first: frontend contracts should remain stable during initial integration.
- Local-first: prefer free/local inference when quality is sufficient.
- Separation of concerns: routes, services, agents, retrieval, providers, and repositories remain distinct.
- Evidence-driven complexity: add components only when a concrete requirement or measured bottleneck exists. Do not execute all retrieval strategies on every request merely because hybrid search is available.
- Grounding over hallucination: course-specific facts should prioritize retrieved context and should not claim sources that were not retrieved.

4. Target Architecture



The architecture is intentionally modular. FastAPI handles transport and validation; the Supervisor Agent handles intent and bounded orchestration; the RAG Agent handles grounded educational responses through a selectable retrieval mode; the Step Evaluator handles code and algorithm evaluation without unsafe execution; the Practice Agent handles validated problem workflows; the LLM provider abstraction isolates model/runtime choices; and repositories isolate PostgreSQL access. BM25, vector, and hybrid retrieval are alternative strategies. Hybrid retrieval is not executed for every request: only the hybrid branch performs BM25 + vector retrieval followed by RRF fusion.

4.1 Data Domains

Domain	Primary persistence
Users	users
Conversations	conversations, messages, optional conversation_summaries
Learning content	concepts, problems
Practice	submissions, progress
Knowledge base	knowledge_documents, knowledge_chunks, pgvector embedding
Observability	application logs/metrics; do not persist secrets or chain-of-thought

4.2 Retrieval Modes

Mode	Best-fit use	Implementation
BM25	Exact terminology, symbols, function/class names, phrase matches	pg_search / ParadeDB
Vector	Conceptual and paraphrased questions	pgvector semantic search
Hybrid	General or uncertain questions where lexical + semantic evidence is useful	BM25 + vector -> RRF; executed only for hybrid requests

4.3 Model Configuration

```

SUPERVISOR_MODEL=qwen3:4b
RAG_AGENT_MODEL=qwen3:8b
STEP_EVALUATOR_MODEL=qwen3:8b
PRACTICE_MODEL=qwen3:8b
EMBEDDING_MODEL=qwen3-embedding:4b
RAG_DEFAULT_MODE=hybrid
RAG_TOP_K=8

```

These are documented candidates from the supplied specification, not mandatory downloads. The implementation must inspect available hardware and locally installed models before setting defaults. Model selection remains configurable.

5. Implementation Strategy and Principles

1. Inspect the repository and produce an implementation report before creating a replacement backend.
2. Preserve existing working integrations and only refactor/replace when there is a concrete technical reason.
3. Build the vertical slice first: PostgreSQL -> knowledge ingestion -> retrieval strategy selection -> Supervisor -> RAG Agent -> LLM Service -> /api/chat. The retrieval layer must support BM25, vector, and hybrid modes as alternatives; hybrid mode invokes BM25 + vector + RRF only when selected.
4. Integrate the frontend through the existing lib/api.js abstraction to minimize UI rewrites.
5. Use strict schemas at API and agent boundaries.
6. Add deterministic and integration tests before adding advanced infrastructure.
7. Keep synchronous request/response as the initial chat transport; add streaming after stability is demonstrated.
8. Gate future complexity through measurement, evaluation, and explicit technical justification.

6. Step-by-Step Implementation Plan

Step 1 - Repository and Frontend Contract Inspection

- Review the existing Python backend, dependencies, agents, tools, RAG code, database layer, environment configuration, and installed model/runtime options.
- Map each frontend function to its required backend endpoint and response shape.
- Record existing authentication configuration and determine how identity can be safely verified by Python.
- Produce a short implementation report covering reusable components, gaps, risks, and major decisions.

Step 2 - Backend Foundation and Configuration

- Establish a Python package layout using the repository's existing structure where practical.
- Create FastAPI application startup/shutdown, health endpoint, CORS configuration, and centralized settings.
- Add Pydantic schemas, structured errors, request IDs, timeouts, and environment-driven configuration.
- Create .env.example and setup documentation.

Example configuration pattern:

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_url: str
    ollama_base_url: str = "http://localhost:11434"
    rag_default_mode: str = "hybrid"
    rag_top_k: int = 8

settings = Settings()
```

Step 3 - PostgreSQL, pgvector, and Migrations

- Configure PostgreSQL as the primary database.
- Enable pgvector.
- Create migration workflow using an existing suitable migration layer or SQLAlchemy 2.x + Alembic + psycopg if starting from scratch.
- Create core tables: users, conversations, messages, optional conversation_summaries, concepts, problems, submissions, progress, knowledge_documents, knowledge_chunks.
- Prefer HNSW for vector indexing unless benchmark evidence indicates IVFFlat is better.

Deliverable: reproducible schema and migration commands.

Step 4 - Knowledge Document Ingestion

- Establish a reproducible ingestion command rather than ingesting on API startup.
- Load supported Tree DSA content, clean and normalize it, preserve source/section metadata, and perform heading-aware or semantic chunking.
- Generate embeddings through the configured embedding provider.
- Persist documents, chunks, metadata, and embeddings.
- Validate embedding dimensions against the selected model rather than hardcoding the dimension in multiple places.

```
python -m backend.rag.ingestion
```

Step 5 - Retrieval Layer

- Define a common retriever.search(query, mode, top_k, filters) interface and make mode explicit at the request boundary.
- Implement three retrieval strategies behind one interface: BM25Retriever, VectorRetriever, and HybridRetriever. They are selectable per request, not three mandatory sequential stages.
- Normalize all results to a RetrievalResult schema so agents never depend on SQL rows.
- Implement RRF inside HybridRetriever only. The hybrid strategy runs BM25 and vector retrieval, combines their ranked results, and applies Reciprocal Rank Fusion before returning the final normalized ranking.
- Build a small evaluation dataset containing query, expected chunk/source, topic, and the intended retrieval mode where known.
- Measure Recall@K, MRR, precision where meaningful, and latency across BM25, vector, and hybrid modes before considering more advanced ranking.

Example normalized result:

```
class RetrievalResult(BaseModel):
    chunk_id: str
    content: str
    score: float
    rank: int
    source: str
    title: str
    metadata: dict
```

Example retrieval selection logic (illustrative):

```
def retrieve(query: str, mode: str, top_k: int):
    if mode == "bm25":
        return bm25.search(query, top_k)
    if mode == "vector":
        return vector.search(query, top_k)
    if mode == "hybrid":
        return rrf(bm25.search(query, top_k), vector.search(query, top_k))
    raise ValueError("Unsupported retrieval mode")
```

Implementation note: RRF is rank-based fusion. A common formulation is $\text{score}(d) = \sum \text{over retrievers of } 1 / (k + \text{rank}_i(d))$; the smoothing constant should remain configurable and be evaluated rather than treated as a universal project constant.

Step 6 - LLM Provider Abstraction

- Create a centralized model registry and provider service.
- Keep agent code independent of provider-specific HTTP calls.
- Support Ollama as the preferred local-first provider and preserve existing OpenRouter/Gemini integration only when already working and useful.
- Provide availability diagnostics without automatically downloading large models.

Example abstraction:

```
class LLMService:
    def generate(self, role: str, messages: list, **kwargs):
        provider = self.registry.provider_for(role)
        return provider.generate(
            model=self.registry.model_for(role),
            messages=messages,
            **kwargs,
        )
```

Step 7 - Supervisor Agent

- Implement a dedicated Supervisor Agent separate from specialized agents.
- Classify intent, choose RAG Agent vs Step Evaluator vs Practice Agent, determine whether retrieval is required, select the retrieval mode when retrieval is required, select tools where necessary, and enforce workflow limits.
- Use structured output with a schema such as SupervisorDecision. Keep the decision operational and do not expose hidden chain-of-thought.
- Enforce maximum steps, timeout, and tool-call limits.
- Avoid exposing chain-of-thought; retain only a short operational reason if useful for diagnostics.

Example decision schema:

```
class SupervisorDecision(BaseModel):
    intent: str
    agent: str # rag_agent | step_evaluator | practice
    needs_rag: bool
    retrieval_mode: str # bm25 | vector | hybrid | none
    tools: list[str] = []
    reason: str
```

Step 8 - RAG Agent

- Implement the RAG Agent as the first production specialized agent.
- Build the educational system prompt around intuition, examples, algorithms, code, complexity, mistakes, hints, and grounded use of retrieved knowledge. The agent should request retrieval through the RAG service rather than embedding database logic inside the agent.
- Construct context from current question + conversation memory + progress context where available + retrieved knowledge. Retrieval mode is selected per request: BM25, vector, or hybrid. Only the hybrid path executes both retrieval strategies and RRF fusion.
- Return ChatResponse with content, sources, and conversation ID. Source metadata must correspond to actually retrieved knowledge.
- Explicitly report knowledge-base insufficiency rather than inventing sources.

Primary milestone: a reliable POST /api/chat path.

Step 9 - Step Evaluator Agent

- Accept code and language, with an optional evaluation/debugging question.
- Perform static syntax-oriented reasoning, logical bug analysis, DSA reasoning, incorrect-assumption identification, hints, suggested fixes, corrected-code guidance, and complexity analysis. Where course-specific knowledge is needed, the Supervisor may route the request through the RAG Agent or retrieval service.
- Return the frontend-compatible response fields currently expected by the /debugger UI: found, line, title, explanation, fix, concept, and complexity. The UI route and frontend function can remain unchanged while the backend role is represented as Step Evaluator.

- Do not execute untrusted code in FastAPI. The first implementation is analysis-only unless a separately approved isolated execution service exists.
- Plan a separate execution service only if actual code execution is later required, with explicit CPU, memory, filesystem, network, and time controls.

Compatibility requirement: retain current ErrorCard expectations or adapt through an explicit frontend adapter.

Step 10 - Practice Agent and Problem APIs

- Implement problem retrieval before relying on generation.
- Validate generated problems with Pydantic.
- Maintain fields required by the frontend including title, difficulty, tags, description, examples, constraints, hints, and starterCode.
- Implement GET /api/problems, GET /api/problems/{id}, and POST /api/problems/{id}/submit.
- Do not claim hidden test execution unless a safe evaluator exists.

Submission result shape must preserve results[] plus passed for initial integration.

Step 11 - Conversation Memory and Progress

- Store conversations and messages in PostgreSQL.
- Keep current frontend-compatible conversation type values during initial integration if they already exist; internally, document their target roles as RAG Agent, Step Evaluator, and Practice Agent rather than forcing an unnecessary contract migration.
- Add summary memory only after threshold-based evidence that long conversations justify it.
- Track initial progress metrics: problems attempted/solved, accuracy, concept progress, recent activity, and streak.
- Keep user memory logically separate from RAG knowledge.

Deliverables: conversation API, progress service, and dashboard data adapter.

Step 12 - Concept APIs and Tree Visualization Data

- Implement GET /api/concepts and GET /api/concepts/{slug}. These endpoints remain domain-content APIs; the frontend may continue to display Tree concepts and use TreeVisualizer.
- Return concept metadata and optional details such as properties, visualization data, examples, code, complexity, mistakes, and related practice problems.
- Reuse the frontend TreeVisualizer; the backend should return structured tree data only when needed.

Step 13 - Authentication, Security, and Error Handling

- Integrate the Python API with the existing NextAuth/Auth.js identity mechanism after reviewing the actual production configuration.
- Do not trust a client-supplied user_id as authorization.
- Configure production CORS to the actual frontend origin.
- Validate payloads, limit request sizes, apply timeouts, parameterize SQL, protect secrets, and return consistent error objects.
- Return 503 for unavailable model/database dependencies where appropriate.

Step 14 - Frontend Integration

- Replace mock bodies inside lib/api.js with fetch-based calls to the Python API.

- Keep route/component changes minimal.
- Adapt snake_case backend responses to current camelCase frontend expectations only at the API boundary when necessary.
- Remove or progressively deprecate mock data as each API becomes authoritative.

Example API call:

```
export async function sendChatMessage(message, history = [], conversationId = null) {
  const response = await fetch(`${API_BASE}/api/chat`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ message, history, conversation_id: conversationId }),
  });

  if (!response.ok) throw new Error("Chat request failed");
  return response.json();
}
```

Step 15 - Testing and Evaluation

- Unit-test supervisor routing, chunking, retrieval strategies, RRF, and schemas.
- Integration-test PostgreSQL, pgvector, BM25 when pg_search is installed, and API validation.
- Agent-test routing with mocked LLM responses.
- Create fixed evaluation sets for concepts, algorithms, debugging, problem generation, and routing.
- Track correctness, grounding, format validity, routing accuracy, and latency.

Step 16 - Observability and Operational Readiness

- Log request ID, endpoint, intent, agent, retrieval mode, retrieved count, model, latency, and error code.
- Never log API keys, passwords, session secrets, authentication tokens, or unnecessary private user data.
- Add health/diagnostic information for database, vector extension, BM25 capability, provider, and configured models.
- Document setup, migrations, ingestion, model configuration, API execution, frontend connection, and troubleshooting.

Deliverable: production-readiness checklist and operational runbook.

Step 17 - Streaming and Future Enhancements

- Only after synchronous chat is stable, introduce SSE/streaming if the frontend experience requires it.
- Evaluate optional reranking only after retrieval evaluation demonstrates a measurable RRF limitation.
- Introduce additional agents and domains incrementally through the agent registry.
- Add safe code execution as a distinct isolated service if product requirements make execution mandatory.

Principle: measured need precedes infrastructure expansion.

7. Agentic Framework Evaluation: LangGraph vs CrewAI

The project documentation explicitly evaluates both permitted agentic implementation choices. The comparison below reflects current official framework capabilities and then maps them to the DSA Coach (Tree) target architecture. It is not a claim that one framework is universally superior.

Dimension	LangGraph	CrewAI
Core model	Low-level graph/state orchestration for long-running, stateful agents; supports	Higher-level multi-agent model using agents, tasks, crews and flows; emphasizes role-

Dimension	LangGraph	CrewAI
	deterministic + agentic workflows	based collaboration plus structured flows
Control flow	Explicit graph nodes, edges, conditional routing and state transitions	Flows provide explicit event-driven routing; Crews provide autonomous collaboration
State / persistence	First-class checkpointing and persistent state; PostgreSQL checkpointer available in reference docs	Flows support state, persistence and resumability; memory/knowledge are integrated at framework level
Human-in-the-loop	First-class interrupts and human intervention patterns	Supported through task/process capabilities and workflow triggers
Memory fit	Strong fit for thread-scoped state and long-running resumable workflows	Strong fit for agent/team memory and workflow state, especially in higher-level abstractions
Complexity	Higher learning curve; more architectural decisions remain with the implementation team	Lower entry friction for role/task based multi-agent systems
Fine-grained orchestration	Excellent	Good to excellent when using Flows, but higher-level Crews can be more opinionated
Rapid prototyping	Moderate	Strong
Enterprise workflow fit for this project	Strong because supervisor routing, bounded steps, persistence and RAG branches can be explicit	Strong, especially if orchestration is expressed as Flows with Crews for autonomous subtasks
Risk for this project	Could become over-engineered if implemented as a large graph for simple routing	Could become too autonomous/role-centric if Crews are used where deterministic routing is preferable
Recommended usage in DSA Coach (Tree)	Primary orchestration candidate	Viable alternative; especially attractive for a faster high-level multi-agent implementation

7.1 LangGraph - Advantages

- Explicit graph/state representation maps directly to Supervisor routing, specialized-agent delegation, retrieval-mode branches, bounded loops, and controlled termination.
- Built-in persistence/checkpointing supports conversational state, resumable execution, and fault recovery when a checkpointer is configured.
- Deterministic and agentic steps can coexist in the same graph, which fits routing, retrieval selection, structured validation, and bounded tool use.
- Official documentation provides a PostgreSQL checkpointer integration, which can align with the project's PostgreSQL-first architecture without introducing a second primary database.
- Fine-grained control over nodes, edges, state, interrupts, and execution flow makes operational policies easier to encode explicitly.

7.2 LangGraph - Disadvantages

- Lower-level orchestration requires more explicit state, node, edge, and error-handling design from the engineering team.
- The abstraction is less opinionated than a role/task framework, so the team carries more responsibility for graph architecture, state modeling, and testing.
- For a genuinely simple single-agent workflow, a graph can add unnecessary ceremony; in this project the graph should therefore remain intentionally small.

- Adopting additional LangChain ecosystem components is optional and should not be treated as a prerequisite unless the chosen implementation actually needs them.

7.3 CrewAI - Advantages

- High-level role/task abstractions map naturally to RAG Agent, Step Evaluator, Practice Agent, and Supervisor concepts.
- Crews provide collaborative multi-agent execution, while Flows provide structured event-driven orchestration and state management.
- Flows are particularly relevant when the application requires deterministic routing, conditional logic, state, persistence, and resumable workflows; Crews are useful when a bounded task benefits from more autonomous collaboration.
- The higher-level abstractions can reduce initial implementation friction for role-oriented multi-agent prototypes.
- Current documentation describes tools, guardrails, memory, knowledge, human-in-the-loop capabilities, and Pydantic-based structured outputs within the framework.

7.4 CrewAI - Disadvantages

- Using Crews as the main routing mechanism could introduce more autonomous collaboration than the initial Supervisor -> specialized-agent requirement needs; Flows are the more appropriate CrewAI primitive for deterministic routing.
- Precise state transitions, persistence, and conditional routing should be modeled through Flows rather than relying on autonomous Crew behavior alone.
- Using both Crews and Flows creates an additional governance decision; the implementation should define one primary orchestration path instead of mixing abstractions without need.
- CrewAI must still be constrained by the application's own architecture: API routes, services, repositories, retrieval, and provider adapters remain independently testable and replaceable.

7.5 Decision Criteria for TreeTutor AI

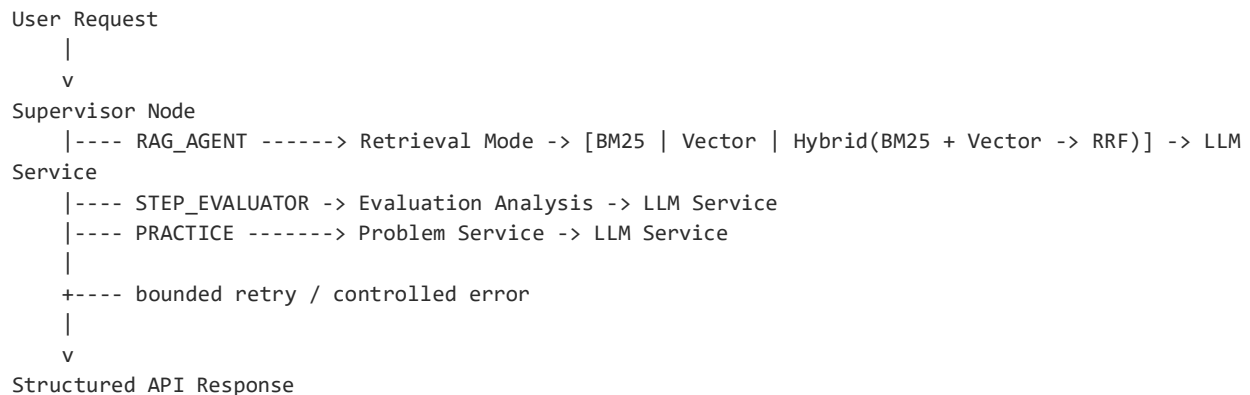
Criterion	Weight	Why it matters
Routing/control precision	High	Supervisor must delegate predictably and choose RAG mode without uncontrolled loops.
Persistent state	High	Conversation continuity and future multi-step workflows require durable state.
PostgreSQL alignment	High	Project mandates PostgreSQL as the primary database.
Educational workflow transparency	High	The system must remain explainable at the orchestration level without exposing chain-of-thought.
Implementation complexity	Medium	The project should remain scalable but not over-engineered.
Ease of extending future agents	Medium	Future domains should be added without rewriting the core.
Operational maturity	Medium	Testing, observability, bounded execution, and recoverability matter for enterprise readiness.

8. Recommended Framework for DSA Coach (Tree)

Recommended choice: LangGraph as the primary agent orchestration framework, using a deliberately small state graph and without introducing an additional orchestration framework unless a concrete requirement emerges.

The primary reason is architectural fit. The target system requires a separate Supervisor Agent that identifies intent, selects RAG Agent vs Step Evaluator vs Practice Agent, decides whether retrieval is required, chooses BM25/vector/hybrid when retrieval is required, supports bounded multi-step work, and returns structured output. LangGraph's official documentation positions it as a low-level orchestration framework with explicit state, nodes, edges, durable execution, persistence, and interrupt/resume capabilities. PostgreSQL checkpoint support is also documented, which aligns with the project's database direction.

CrewAI remains a credible alternative, particularly when the team values faster role/task composition or wants Flows for deterministic orchestration with Crews for autonomous subtasks. For the initial DSA Coach (Tree) architecture, however, the requirements emphasize explicit routing, bounded state transitions, and predictable retrieval branching more than autonomous agent collaboration. A minimal LangGraph implementation is therefore recommended.



Important governance point The framework should implement orchestration, not become the application architecture. FastAPI, services, repositories, RAG, model providers, and frontend contracts remain independently testable and replaceable.

9. Testing and Quality Assurance

9.1 Unit Tests

- Supervisor routing
- BM25 retrieval strategy
- Vector retrieval strategy
- RRF hybrid ranking
- Chunking and metadata preservation
- Pydantic schemas and serializers
- Provider/model registry resolution
- Frontend response adapters where applicable

9.2 Integration Tests

- PostgreSQL connection and migrations
- pgvector query and index behavior
- BM25 query when pg_search is installed
- API validation and error contracts
- Authentication verification in the target deployment configuration

9.3 Agent Evaluation

Evaluation area	Examples of measures
RAG Agent	correctness, grounding, helpfulness, source fidelity
Step Evaluator	bug identification accuracy, fix quality, contract validity
Practice	problem validity, difficulty correctness, schema validity
Supervisor	routing accuracy, unnecessary delegation rate, bounded execution
Retrieval	Recall@K, MRR, precision where meaningful, latency

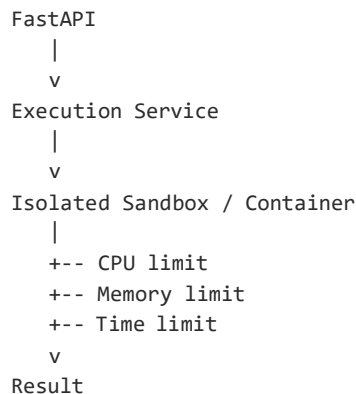
The supplied specification emphasizes measurement before adding advanced retrieval components. In particular, hybrid/RRF should be evaluated against BM25 and vector retrieval rather than assumed to be best by default.

10. Security, Governance, and Observability

10.1 Security Controls

- Secrets remain in environment variables and are excluded from source control.
- API request payloads are validated with strict schemas.
- Request body sizes and upstream/model timeouts are bounded.
- No arbitrary user code is executed in the FastAPI process.
- Authentication is verified from the authenticated request context rather than a client-supplied user_id.
- Database access uses parameterized SQL/ORM patterns.
- Production CORS is restricted to the actual frontend origin.
- Internal prompts, API keys, stack traces, and chain-of-thought are never exposed to the frontend.

10.2 Code Execution Boundary



The first version should not execute arbitrary student code directly. Actual execution belongs behind an isolated execution boundary and should be introduced only with an approved sandbox architecture.

10.3 Observability

Log / metric	Purpose
request_id	Trace a single API request end-to-end
endpoint	Identify service path

Log / metric	Purpose
intent / agent	Understand routing behavior
retrieval_mode / retrieved_count	Assess RAG behavior
model	Track model configuration used
latency	Measure performance
error code	Monitor failure patterns

Sensitive values such as API keys, passwords, session secrets, authentication tokens, and unnecessary private user data must not be logged.

11. Deployment and Operational Readiness

The local-first development target is conceptually PostgreSQL + pgvector + pg_search + Ollama + Python API. Docker may be used when it materially improves reproducibility, particularly for PostgreSQL and required extensions. The specification explicitly discourages adding infrastructure solely because it is fashionable.

11.1 Environment Configuration

```

APP_ENV=development
API_HOST=0.0.0.0
API_PORT=8000
DATABASE_URL=postgresql://...
OLLAMA_BASE_URL=http://localhost:11434
SUPERVISOR_MODEL=qwen3:4b
RAG_AGENT_MODEL=qwen3:8b
STEP_EVALUATOR_MODEL=qwen3:8b
PRACTICE_MODEL=qwen3:8b
EMBEDDING_MODEL=qwen3-embedding:4b
RAG_DEFAULT_MODE=hybrid
RAG_TOP_K=8
FRONTEND_URL=http://localhost:3000

```

11.2 Diagnostic Checks

- Configured provider and model
- Configured model availability
- PostgreSQL availability
- pgvector availability
- BM25/pg_search capability availability
- Embedding model readiness

12. Current vs Future Roadmap

Phase	Primary outcome	Scope
Phase 0	Assessment	Inspect repository, frontend contracts, models, backend code, and auth configuration.
Phase 1	Core RAG vertical slice	PostgreSQL + pgvector + BM25 + ingestion + selectable retrieval modes + hybrid/RRF + RAG Agent + Supervisor + /api/chat.
Phase 2	Learning features	Step Evaluator, Practice, problem APIs, Concepts,

Phase	Primary outcome	Scope
		conversation retrieval.
Phase 3	Learning intelligence	Progress, optional summaries, personalization foundations.
Phase 4	Production enhancement	Streaming, safe execution, advanced retrieval evaluation, optional reranking, scaling based on measured demand.
Phase 5	Domain expansion	Graph Tutor, DP Tutor, Linked List Tutor, Stack/Queue, Sorting, and other DSA domains.

13. Definition of Done and Acceptance Criteria

13.1 First Meaningful Backend Milestone

9. PostgreSQL is operational.
10. pgvector is operational.
11. BM25 capability is operational or a clearly approved fallback has been documented.
12. Ollama/local LLM execution is operational.
13. The embedding model is operational.
14. Tree knowledge can be ingested reproducibly.
15. BM25 retrieval works.
16. Vector retrieval works.
17. Hybrid/RRF retrieval works.
18. Supervisor routing works.
19. RAG Agent responds with grounded output.
20. POST /api/chat works end-to-end against the frontend contract.

13.2 Acceptance Criteria for Frontend Integration

Frontend function	Backend endpoint	Initial acceptance criterion
sendChatMessage()	POST /api/chat	Real backend response replaces mock and conversation_id is preserved.
debugCode()	POST /api/debug	Step Evaluator response matches the current ErrorCard contract.
fetchProblems()	GET /api/problems	Problem list renders without mock data.
submitSolution()	POST /api/problems/{id}/submit	Response preserves results[] and passed fields; no false claim of hidden execution.
fetchConcepts()	GET /api/concepts	Concept library loads from backend.
fetchConversation()	GET /api/conversations/{id}	Stored conversation renders correctly.

14. Risks, Constraints, and Mitigations

Risk / constraint	Impact	Mitigation
BM25 extension unavailable	Retrieval requirements cannot be met as specified	Validate pg_search availability early; do not silently relabel PostgreSQL FTS as BM25.
Local model too large	High latency / instability	Inspect hardware and installed models; use

Risk / constraint	Impact	Mitigation
		smaller candidate model where required.
Authentication verification ambiguous	Security exposure	Stop before production auth integration and base verification on actual Auth.js configuration.
Agent loops / runaway tools	Latency and cost risk	Enforce max steps, timeout, and tool-call limits.
RAG quality insufficient	Incorrect educational answers	Evaluate BM25/vector/hybrid separately; improve chunking and metadata before considering a reranker.
Unsafe code execution	Critical security risk	No direct exec/eval/subprocess execution; use a separate sandbox only after approval.
Over-engineering	Delivery and maintenance cost	Use one primary agent framework, one database, one vector layer, and only measured infrastructure.
Frontend contract drift	Integration breakage	Keep lib/api.js as the compatibility boundary; retain /debugger and /api/debug during initial integration even though the backend role is named Step Evaluator.

15. Conclusion

DSA Coach (Tree) should be implemented as a specialized educational GenAI platform rather than a generic chatbot. TreeTutor AI remains the existing frontend application. The implementation should preserve the existing Next.js experience while introducing a disciplined Python backend built around FastAPI, PostgreSQL, pgvector, true BM25, selectable retrieval modes with hybrid/RRF, configurable local-first models, and a dedicated Supervisor Agent with specialized RAG Agent, Step Evaluator, and Practice Agent workflows.

The implementation sequence deliberately prioritizes the smallest production-worthy vertical slice: knowledge ingestion, retrieval strategy selection, Supervisor, RAG Agent, and POST /api/chat. Once that path is reliable, Step Evaluator, Practice, Conversations, Concepts, and Progress can be layered in without weakening architectural boundaries. The recommended agent framework is LangGraph because its explicit stateful orchestration model and persistence capabilities align closely with the project's controlled Supervisor architecture; CrewAI remains a viable alternative, especially when higher-level role/task abstractions or Flow-plus-Crew composition offer meaningful delivery advantages.

The guiding enterprise principle is controlled evolution: every major dependency or infrastructure addition should have a concrete role, measurable benefit, and clear operational boundary. This keeps DSA Coach (Tree) scalable without making version 1 unnecessarily complex.

Appendix A - Example API and Agent Contracts

A.1 Chat Request

```
{
  "message": "What is the difference between a binary tree and a BST?",
  "history": [],
  "conversation_id": null
}
```

A.2 Chat Response

```
{
  "id": "m123",
  "role": "assistant",
  "content": "A binary tree...",
  "sources": ["Binary Search Tree", "BST Traversal"],
  "conversation_id": "c123"
}
```

A.3 Step Evaluator Response

```
{
  "found": true,
  "line": 6,
  "title": "Missing +1 for the current node",
  "explanation": "...",
  "fix": "return max(left, right) + 1",
  "concept": "Binary Tree Traversal",
  "complexity": {"time": "O(n)", "space": "O(h)"}
}
```

A.4 Retriever Interface

```
retriever.search(
  query=query,
  mode="hybrid", # or "bm25" / "vector"; one mode per request
  top_k=8,
  filters=None,
)
```

Appendix B - Reference Sources

Project-provided sources:

- BACKEND_SPECIFICATION_CODEX.md - supplied implementation specification and frontend/backend contract.
- treetutor-ai.zip - supplied frontend source archive, including routes, components, lib/api.js, mock data, and Auth.js scaffold.

Supplementary framework research:

- LangGraph official documentation - overview, graph/state orchestration, durable execution, persistence, and agent workflow capabilities. <https://docs.langchain.com/oss/python/langgraph/overview>
- LangGraph official documentation - persistence, checkpointing, memory, and PostgreSQL checkpoint integration. <https://docs.langchain.com/oss/python/langgraph/persistence>

- CrewAI official documentation - Agents, Crews, Flows, Tasks/Processes, guardrails, memory, knowledge, structured outputs, persistence, and resumable workflows. <https://docs.crewai.com/>

RRF technical reference: Elastic documentation describes RRF as a rank-based fusion method that combines independently ranked result sets using a reciprocal-rank formula.

<https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion>

Note: Framework capabilities and terminology in Appendix B were re-checked against the official documentation available on 13 August 2026. Product-specific scope, interfaces, and architecture decisions are derived from the supplied project materials. The retrieval-mode policy is derived from the supplied implementation specification and clarified here to make the request-level branching explicit.